

제조데이터 분석과 최적화

제조 최적화(2)

김한진

gks359@cbnu.ac.kr

충북대학교 산업인공지능연구센터

CONTENTS



스케줄링 이론



자원/배합 최적화 vs 스케줄링



스케줄링 문제의 구현



스케줄링 문제

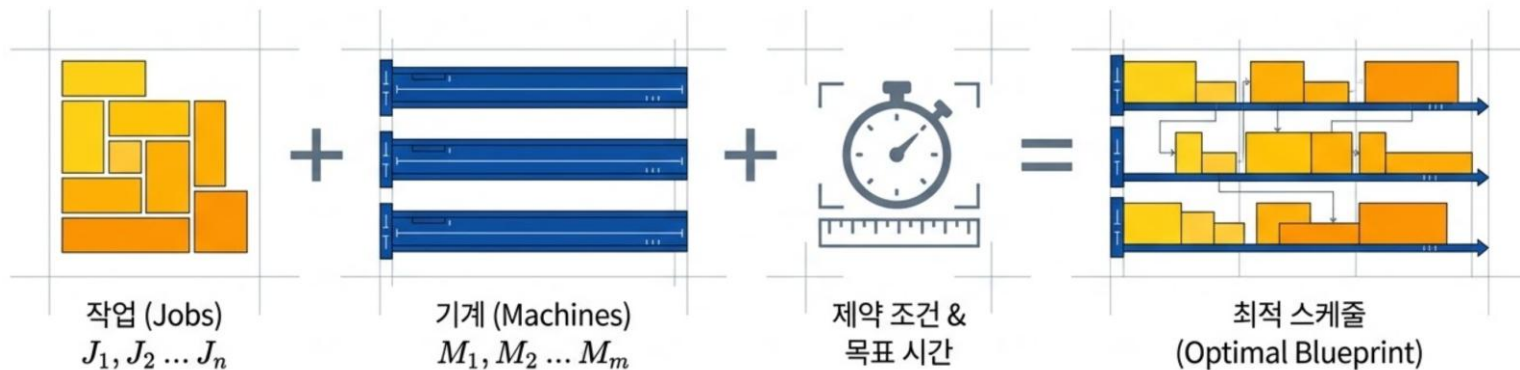
■ 스케줄링 이론

➤ 스케줄링 문제?

n개의 작업(Jobs)를 m개의 기계(Machines)에 할당하여, 주어진 제약 조건을 만족하면서 목표 함수를 최적화(Minimize/Maximize)하는 수학적 의사결정 과정

작업(Job) : 최종적으로 완성되어야 하는 대상

공정(Operation) : 해당 작업을 완성하기 위해 특정 기계에서 수행해야 하는 개별 단위 업무

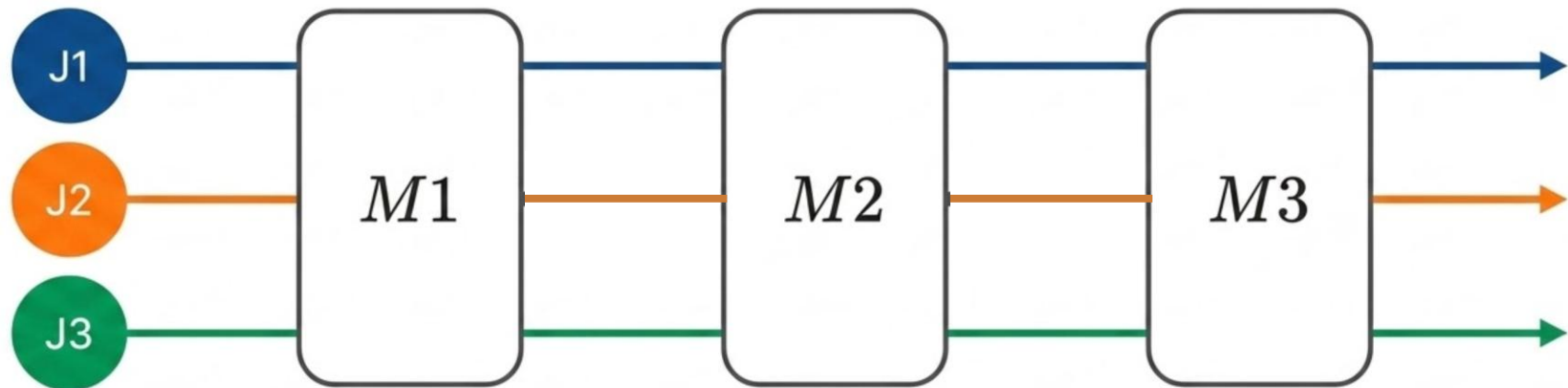


■ 스케줄링 이론

➤ Flow shop, 플로우샵

m 개의 기계가 직렬로 배치된 환경. 모든 작업은 m 개의 기계에서 각각 처리되어야 하며, 모두 동일한 경로를 따라야 함

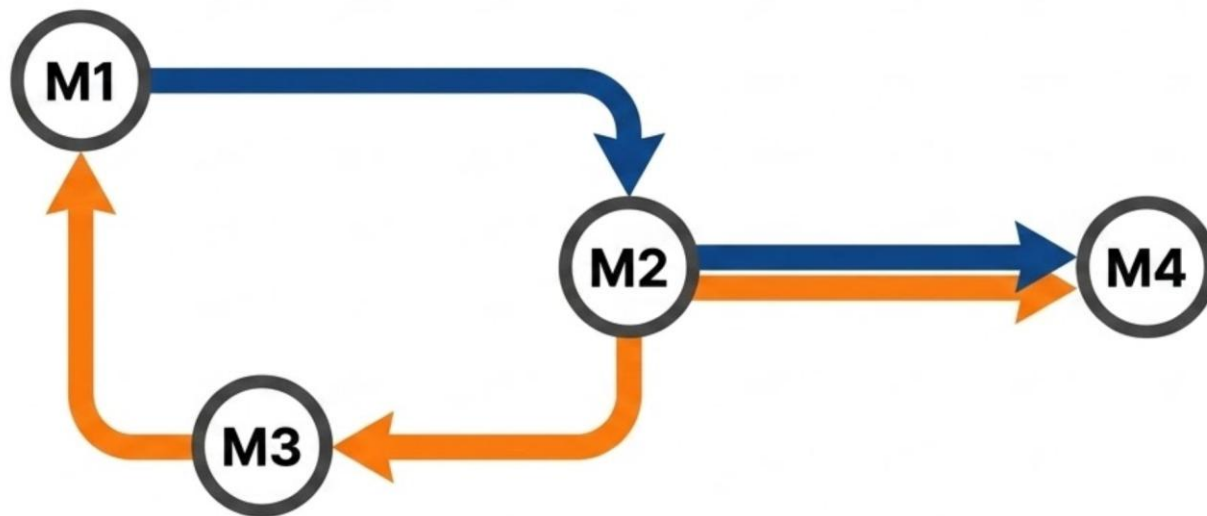
✓ $m1 \rightarrow m2 \rightarrow m3$ 로 정해진 순서대로만 이동 가능



■ 스케줄링 이론

➤ Job shop, 잡 샵

- m개의 기계가 있는 환경에서 각 작업이 고유하고 미리 정해진 경로를 따라야 하는 환경
✓ 작업마다 경로는 다르지만, 스케줄러가 임의로 경로를 바꿀 수는 없음

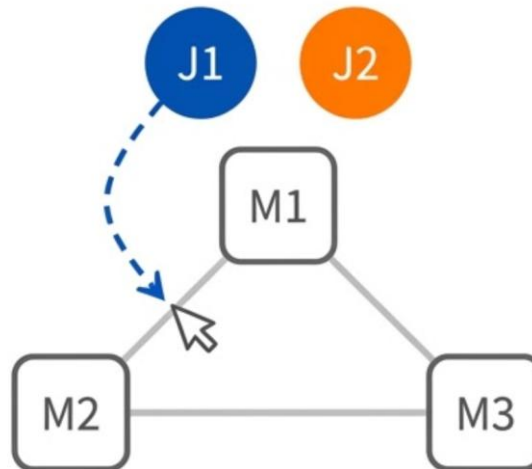


■ 스케줄링 이론

➤ open shop, 오픈샵

m 개의 기계가 있으며, 각 작업은 모든 기계에서 각각 처리되어야 하지만, 기계들 사이의 작업 경로에는 아무런 제한이 없는 환경

- ✓ 방향성 없는 그래프와 같음. 스케줄러가 가장 효율적인 순서를 찾아 직접 화살표(방향)를 그려 넣어야 함



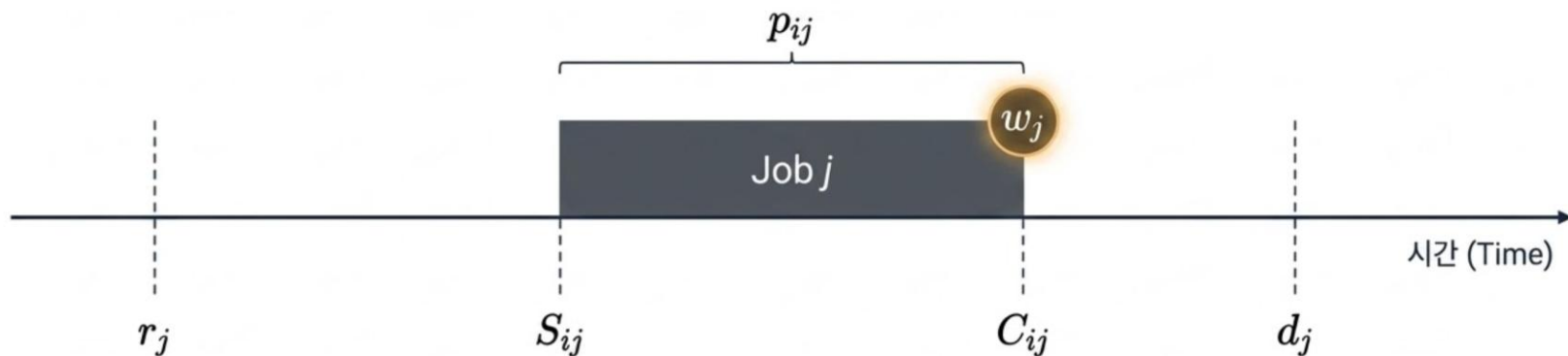
■ 스케줄링 이론

➤ 스케줄링 모델 비교

구분	플로우 샵 (Flow Shop)	잡샵 (Job Shop)	오픈 샵 (Open Shop)
라우팅 특성	모든 작업이 예외 없이 완전히 동일한 기계 방문 순서를 따름 (일방향 흐름)	작업마다 서로 다른 고유한 기계 방문 순서가 정해져 있음 (복잡하게 얹히는 경로)	거쳐야 하는 기계 목록만 존재할 뿐, 기계 방문 순서는 완전히 자유로움
스케줄러의 권한 및 유연성	라인 맨 처음 기계에 어떤 작업을 먼저 투입할지 그 순서만 결정 (경로나 흐름 자체에는 개입할 수 없어 권한이 가장 제한적임)	기계별로 대기 중인 작업 중 어떤 것을 먼저 처리할지 결정 (단, 개별 작업의 정해진 공정 순서를 임의로 바꿀 수는 없음)	기계별 작업 우선순위뿐만 아니라, 각 작업이 어떤 기계를 먼저 방문할지(라우팅 경로)까지 스케줄러가 자유롭게 결정
주요 제약 조건 (Constraints)	1. 모든 작업에 동일한 선후행 제약 2. 기계 용량 및 비선점 제약 3. (순열형의 경우) 추월 불가 제약	1. 개별 작업별 선후행 제약 2. 기계 용량 제약 (동시에 1개만) 3. 작업 비선점 제약 (중단 불가)	1. 작업 동시 처리 불가 2. 기계 용량 제약 3. 비선점 제약
대표 산업 사례 (Examples)	자동차/스마트폰 컨베이어 조립 라인, 화학/식료품 연속 가공 공정, 소품종 대량 생산	맞춤형 기계 부품 가공 공장, 맞춤형 가구 제작소, 다품종 소량 생산	종합건강검진센터, 자동차 종합 수리 센터

■ 스케줄링 이론

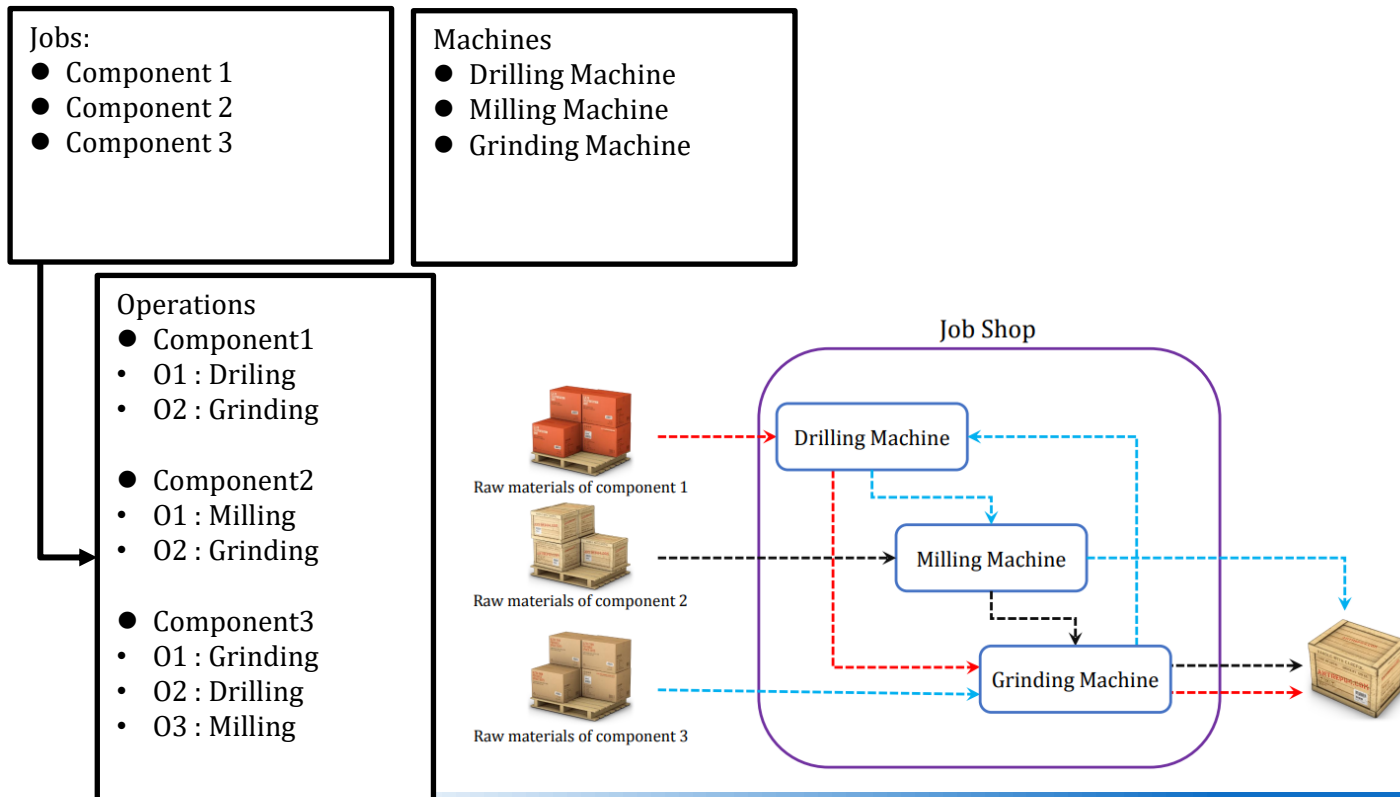
➤ 작업특성과 제약조건



r_j	: 도착 시간 (Release Date) - 작업이 시스템에 도착해 처리가 가능해지는 최초 시점.	C_{ij}	: 완료 시간 (Completion Time) - $C_{ij} = S_{ij} + p_{ij}$ (중단 없을 시).
S_{ij}	: 시작 시간 (Start Time) - 기계 i 에서 작업 j 가공 시작.	d_j	: 납기일 (Due Date) - 반드시 완료되어야 하는 목표 시점.
p_{ij}	: 처리 시간 (Processing Time) - 순수 가공 및 준비(Setup) 시간.	w_j	: 가중치 (Weight) - 작업의 긴급도 및 상대적 중요성.

■ 스케줄링 이론

- Job : 고객의 주문이나 생산되어야 하는 하나의 완성품 단위
- Machine : Job을 가공하기 위해 공장에 배치된 설비나 장비
- Operation 하나의 Job이 완성되기 위해서는 여러 단계의 공정을 순서대로 거쳐야 함.
(스케줄링의 가장 기본 단위)



Yazdani, Maziar, et al. "Optimizing the sum of maximum earliness and tardiness of the job shop scheduling problem." *Computers & Industrial Engineering* 107 (2017): 12-24.

■ 스케줄링 이론

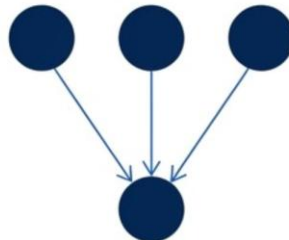
➤ 작업특성과 제약조건

chains



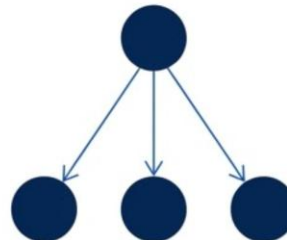
선형 순서
(진입/진출 차수 1 이하)

intree



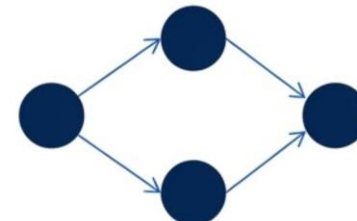
여러 작업이
하나의 결과로 병합

outtree



하나의 작업에서
여러 갈래로 파생

sp-graph

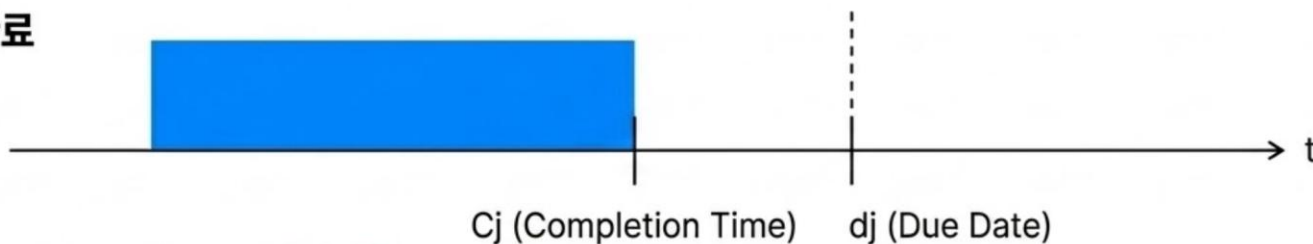


직렬 및 병렬
구성의 결합

■ 스케줄링 이론

➤ 스케줄링의 목적, 목적함수

시나리오 1: 조기 완료



지연 시간 (Lateness): $L_j = C_j - d_j$ (음수 값)

지각 시간 (Tardiness): $T_j = \max(C_j - d_j, 0) = 0$

시나리오 2: 지연 완료



지연 시간 (Lateness): $L_j = C_j - d_j$ (양수 값)

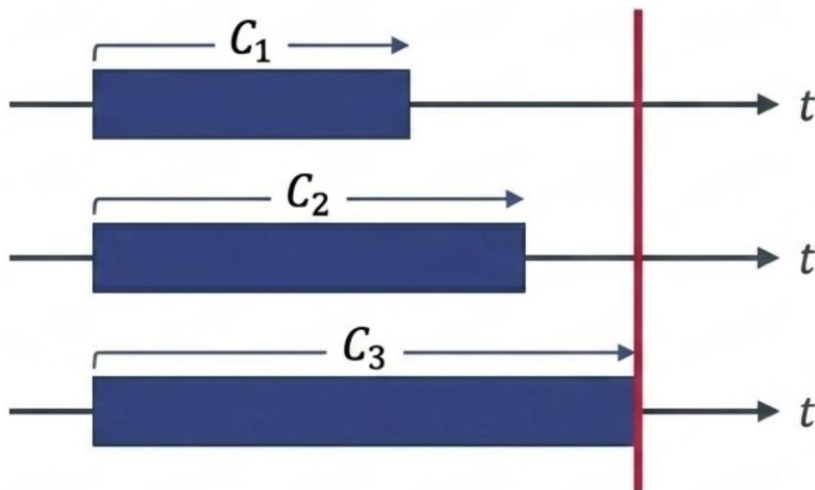
지각 시간 (Tardiness): $T_j = \max(C_j - d_j, 0) = \text{양수 값}$

■ 스케줄링 이론

➤ 스케줄링의 목적, 목적함수

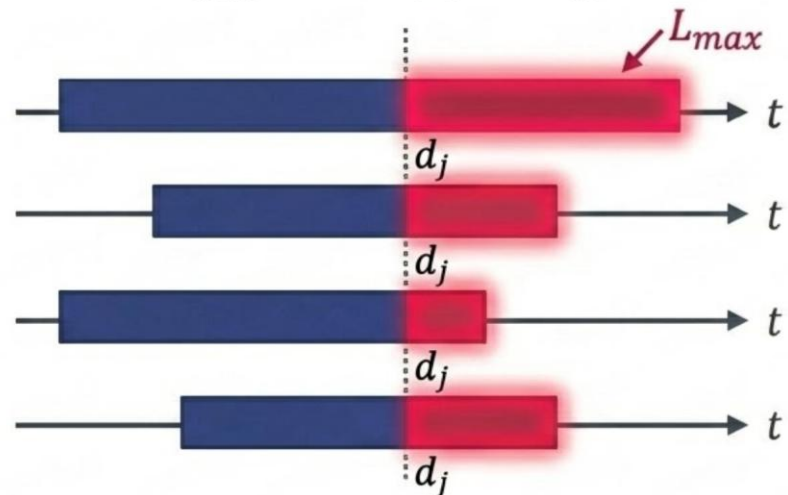
Makespan (C_{max})

$$C_{max} = \max(C_1, \dots, C_n)$$



Maximum Lateness (L_{max})

$$L_{max} = \max(L_1, \dots, L_n)$$



■ 스케줄링 알고리즘

- **Dispatching rules** : 어떤 작업을 먼저 기계나 설비(또는 작업자)에 할당할지 결정하는 우선순위 규칙

FCFS - First Come, First Served →  →

EDD - Earliest Due Date →  →

SPT - Shortest Processing Time →  →

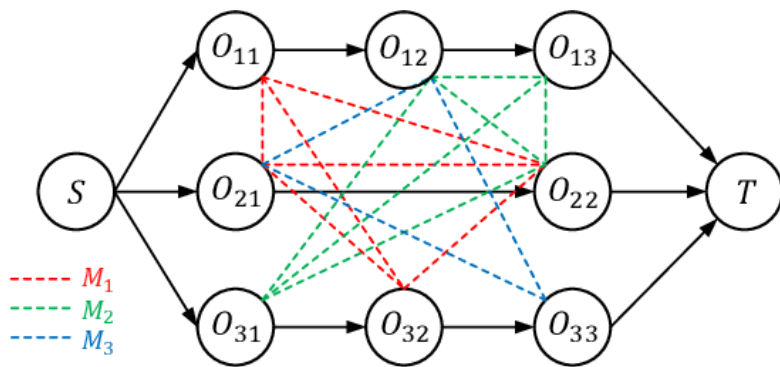
LPT - longest processing time →  →

■ 최신 스케줄링 방법

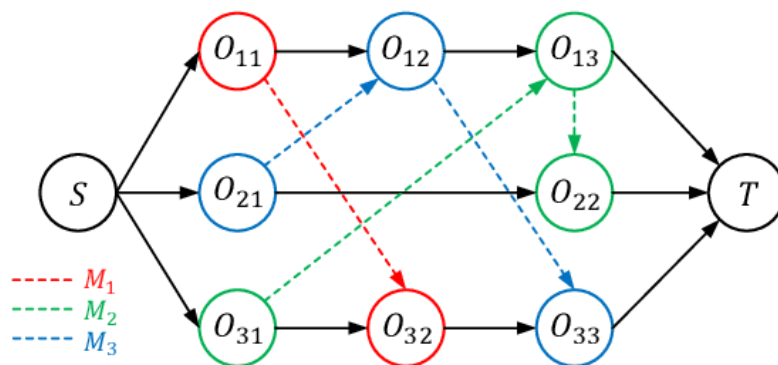
➤ GNN(Graph Neural Network)과 DRL(Deep Reinforcement Learning)을 적용한 Scheduling 기법

Flexible Job shop Scheduling : 하나의 공정을 처리할 수 있는 기계가 여러 대 존재

- Job : 고객의 주문이나 생산되어야 하는 하나의 완성품 단위
- Machine : Job을 가공하기 위해 공장에 배치된 설비나 장비
- Operation 하나의 Job이 완성되기 위해서는 여러 단계의 공정을 순서대로 거쳐야 함.
(스케줄링의 가장 기본 단위)



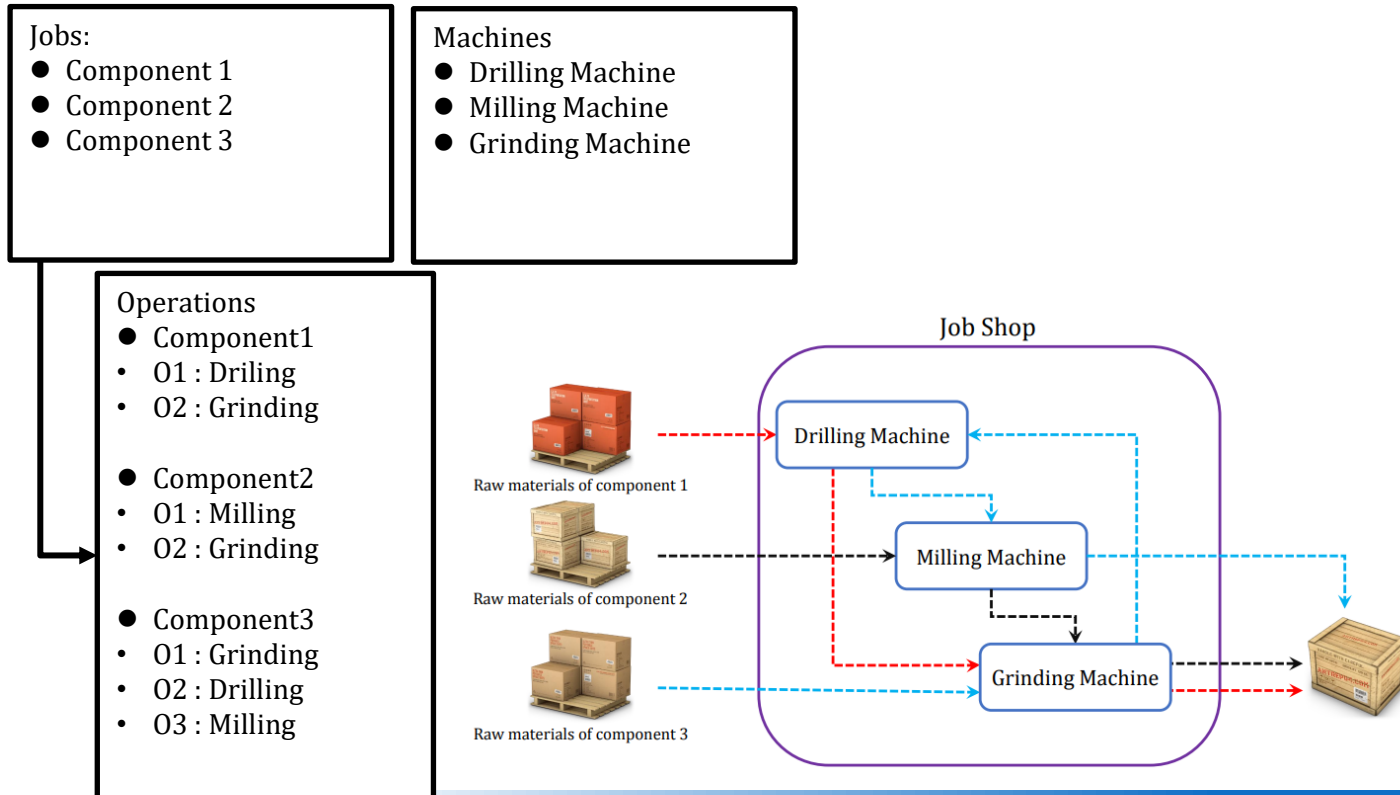
(a) An FJSP instance



(b) A possible solution

■ 최신 스케줄링 방법

- Job : 고객의 주문이나 생산되어야 하는 하나의 완성품 단위
- Machine : Job을 가공하기 위해 공장에 배치된 설비나 장비
- Operation 하나의 Job이 완성되기 위해서는 여러 단계의 공정을 순서대로 거쳐야 함.
(스케줄링의 가장 기본 단위)

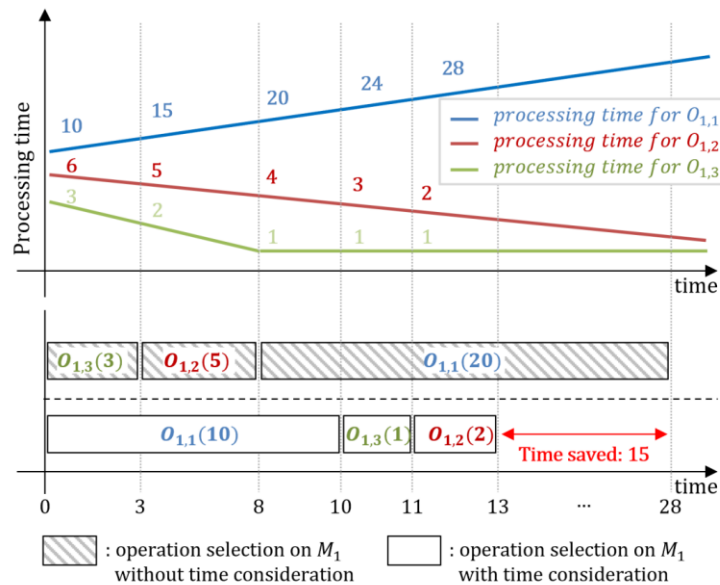


■ 최신 스케줄링 방법

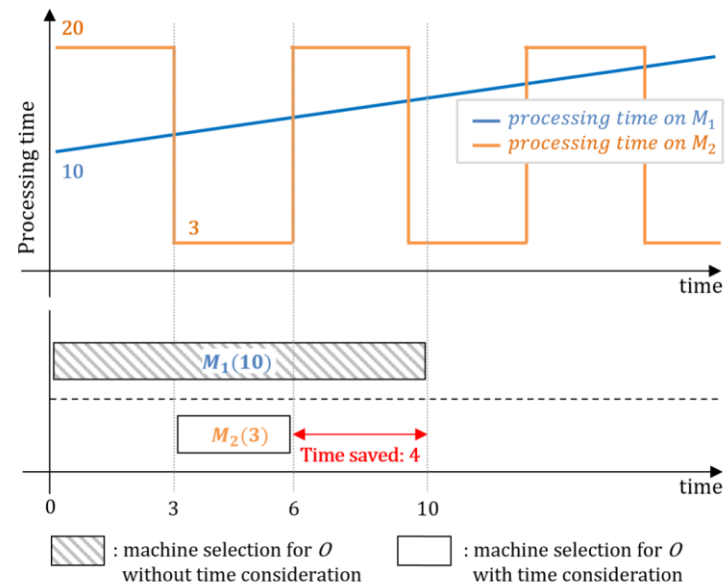
➤ GNN(Graph Neural Network)과 DRL(Deep Reinforcement Learning)을 적용한 Scheduling 기법

Flexible Job shop Scheduling : 하나의 공정을 처리할 수 있는 기계가 여러 대 존재

- 기계 할당 : 각 공정을 수행할 수 있는 여러 기계 중 어떤 기계에 할당할 것인가?
- 작업 순서 결정 : 특정 기계에 여러 작업이 할당되었을 때, 어떤 작업부터 먼저 처리할 것인가?



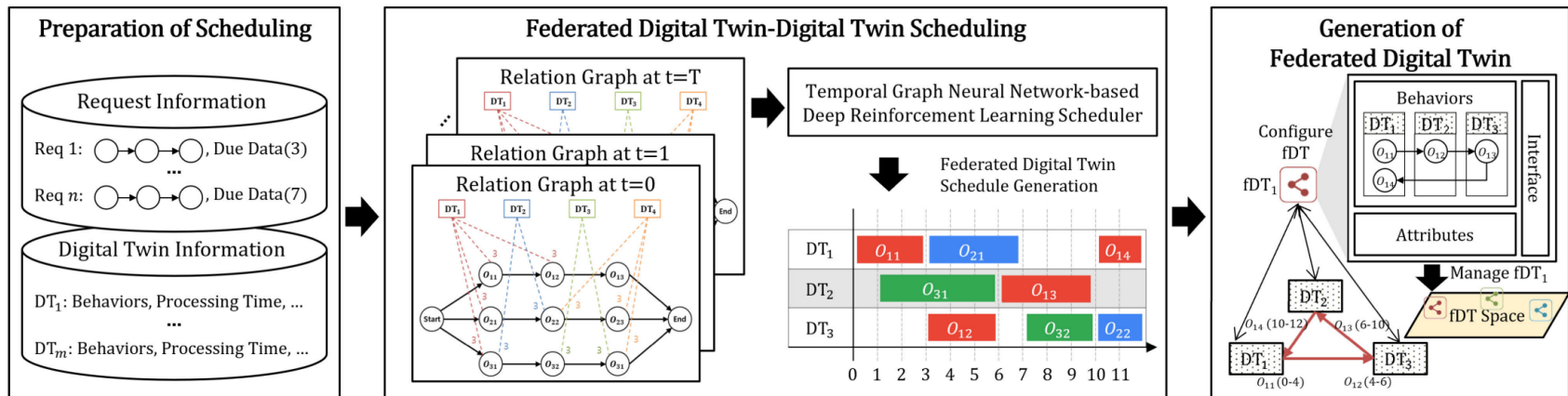
(a) Operation selection when number of machine=1



(b) Machine assignment when number of operation=1

■ 최신 스케줄링 방법

- GNN(Graph Neural Network)과 DRL(Deep Reinforcement Learning)을 적용한 Scheduling 기법

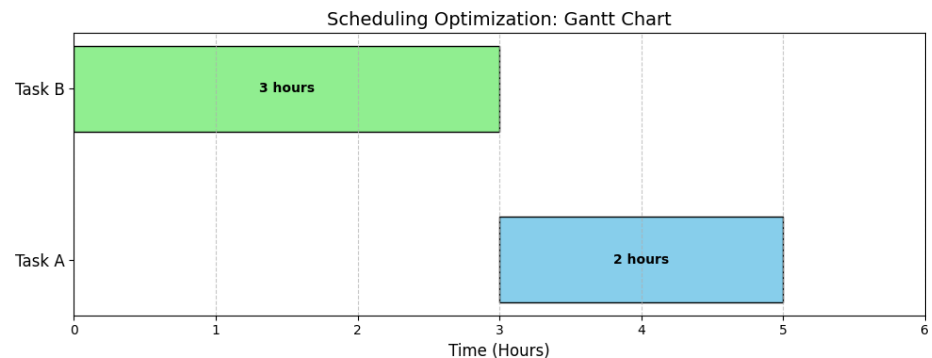
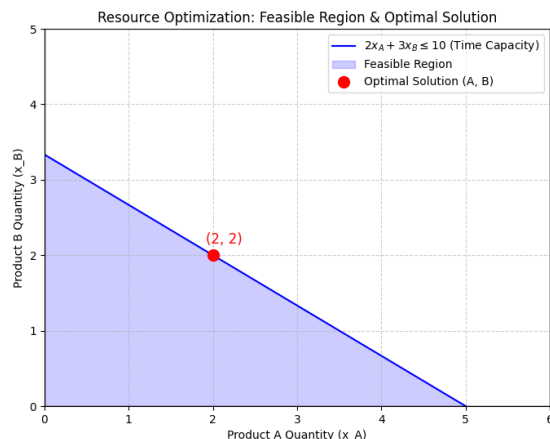


■ 자원/배합 최적화 문제와 스케줄링 문제의 차이

➤ 시간의 역할 : 용량(capacity) vs 순서(sequence)

시간에 대한 취급

- 자원/배합 최적화에서의 시간 (시간 = 총량/바구니) :
시간은 예산이나 원자재와 같은 '소비 가능한 자원'
예) 하루 0 시간의 기계 가동 시간이 주어질 때, 제품 A와 B를 각각 몇 개씩 생산해야 최대 이익?
- 스케줄링에서의 시간 (시간 = 타임라인/흐름) :
시간은 '특정 사건이 일어나는 시점'
예) 제품 A의 조립(작업 1)이 끝나야만 도색(작업 2)을 시작할 수 있음과 같은 선후 관계
→ 총 0 시간의 가동 시간 이내라도, 조립이 끝나기 전 도색 기계가 돌아가는 해는 오답



■ 자원 할당 vs 스케줄링 - 실습코드(resource_vs_scheduling)

➤ 자원 및 배합 최적화

시간의 의미: Capacity (가용 용량)

주요 목표: 이익 최대화 또는 비용 최소화

결정 변수: x (생산량, 투입량 등 '수량'을 나타내는 정수/연속 변수)

제약 조건: 단순한 덧셈 형태의 선형식

$$ax_1 + bx_2 \leq C$$

➤ 자원 및 배합 최적화

시간의 의미: Sequence & Timeline (순서와 흐름)

주요 목표: 전체 소요 시간(Makespan) 최소화, 지각(Tardiness) 최소화

결정 변수:

• t : 작업의 '시작 시점'

• y : '작업 순서'를 정하는 이진 변수 (Binary, 0 또는 1)

제약 조건 (Big-M 기법):

스케줄링은 "A 다음에 B를 하거나, B 다음에 A를 해라"라는 **논리적인 조건** 필요

이를 수학적으로 표현하기 위해 매우 큰 수인 M 을 사용한 'Either-Or(이것 아니면 저것)' 제약식 만들

$$t_B \geq t_A + p_A - M(1 - y)$$

$$t_A \geq t_B + p_B - M \cdot y$$

■ 자원 할당 vs 스케줄링

구분	자원 최적화 (Resource)	스케줄링 (Scheduling)
시간의 개념	소비 가능한 용량 (Capacity)	흐름을 가진 순서 (Sequence)
던지는 질문	"무엇을 얼마나 섞을까?"	"누가 먼저, 언제 시작할까?"
의사결정 변수	수량 x	시작 시간 (t), 순서 (y)
수학적 난이도	덧셈 위주의 직관적인 수식	0과 1의 이진 논리가 포함된 복잡한 수식
대표적인 시각화	2차원 그래프 (실현 가능 영역)	간트 차트 (Gantt Chart)

■ 이산 사건 시뮬레이션 프로그래밍 패러다임

프로그래밍 패러다임	작동 메커니즘 및 철학
활동 중심 Activity- Oriented	시뮬레이션 시간을 아주 미세한 단위(Tick)로 고정 분할하여 점진적으로 진행 매 시간 단계마다 시스템 내의 모든 활동을 전수 검사하여 조건이 충족되는 이벤 트가 있는지 확인
사건 중심 Event- Oriented	시스템의 비활동 시간을 건너뛰고, 예약된 다음 이벤트의 발생 시점으로 시뮬레이 션 시계(Clock)를 직접 이동시키는 '단축(Shortcut)' 기법 사용
프로세스 중심 Process- Oriented	시뮬레이션의 각 활동을 독립적인 프로세스(경량 스레드 또는 코루틴)로 모델링 애플리케이션에 특화된 스레드(예: 고객 도착 스레드)와 이벤트 셋을 관리하는 중 앙 매니저 가 비동기적으로 상호작용

■ Simpy

➤ Python 기반의 프로세스 기반 이산 이벤트 시뮬레이션 프레임워크

```
>>> import simpy
>>>
>>> def clock(env, name, tick):
...     while True:
...         print(name, env.now)
...         yield env.timeout(tick)
...
>>> env = simpy.Environment()
>>> env.process(clock(env, 'fast', 0.5))
<Process(clock) object at 0x...>
>>> env.process(clock(env, 'slow', 1))
<Process(clock) object at 0x...>
>>> env.run(until=2)
fast 0
slow 0
fast 0.5
slow 1
fast 1.0
fast 1.5
```



<https://simpy.readthedocs.io/en/latest/>

■ Simpy

- 자원(simpy.Resource) : 한 번에 접근할 수 있는 프로세스의 숫자가 엄격하게 제한된 설비나 서비스 모델링 - 자원 사용 권한 요청 및 반환 의무
- 컨테이너(simpy.Container) : 개별적인 인스턴스 구분이 필요없는 재화의 흐름을 모델링 - 최대 수용 한계를 지니며, 프로세스가 물질을 추가할 때는 put, 소비할 때는 get
- 스토어(simpy.Store) : 개별적인 객체 인스턴스의 생산과 소비 - 각기 고유한 식별자나 특성 데이터를 지닌 여러 종류의 객체가 복합적으로 저장. Items 속성 사용

자원 (Resource)



- 개별 제한
- 은행 창구
- 주유기

컨테이너 (Container)



- 연속 벌크
- 액체 부피
- 물/가스 탱크

스토어 (Store)



- 개별 객체
- 창고 선반
- 소포/상품

■ Simpy 기능 (함수)



`env.process()`

프로세스 동시 실행. 여러 부품이나
생산 라인이 동시에 가동되는
상황을 제어합니다.



`env.timeout()`

시간의 흐름 모사. 부품 가공이나
이동에 소요되는 절대적인 시간을
시뮬레이션합니다.

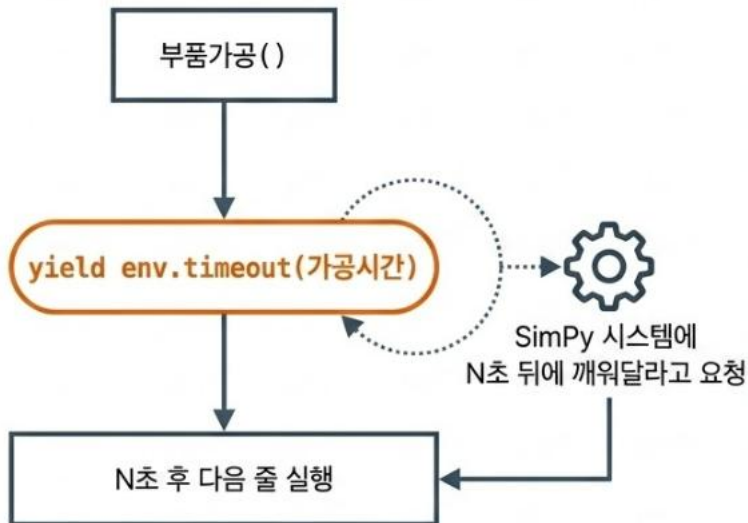


`simpy.Resource`

자원 경쟁과 대기. 한정된 설비나
작업자를 여러 부품이 쟁탈하며 발생하는
'대기열(Queue)'을 모사합니다.

■ Simpy yield

- SimPy에서 yield는 "잠시 멈춤"과 "제어권 넘김"을 의미 (Python 제너레이터 활용)

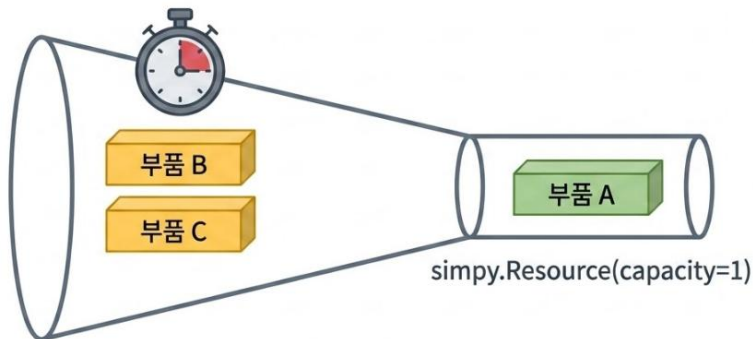


```
yield env.timeout(가공시간) # 가공시간만큼 시간이 흐름  
  
env.process(부품가공(env, '부품A', 3))
```

```
t= 0.0 | 부품A 가공 시작 (3초 소요 예정)  
t= 2.0 | 부품C 가공 완료! ✓  
t= 3.0 | 부품A 가공 완료! ✓
```

■ Simpy resource

- 대기열(Queue) : 한정된 자원(Resource)을 사용하기 위해 순서를 기다리는 사람이나 작업들의 모임



현실의 공장에는 제약이 존재합니다. 설비가 배정될 때까지 부품은 ‘대기열(Queue)’에서 기다려야 하며, 여기서 병목 현상이 시작됩니다.

CODE FOCUS

```
설비 = simpy.Resource(env, capacity=1)

with 설비.request() as req:
    yield req # 설비가 빌 때까지 대기!
```

병목의 시작. 설비가 배정될 때까지 코드는 다음 줄로 넘어가지 않고 대기(yield)합니다.

TERMINAL OUTPUT TRACE

```
t= 0.0 | 부품A 도착 → 설비 요청...
t= 0.0 | 부품A 바로 배정! → 가공 시작
t= 0.0 | 부품B 도착 → 설비 요청...
t= 3.0 | 부품A 가공 완료! ✓
t= 3.0 | 부품B 대기 완료 (⌚ 3.0초 기다림) → 가공 시작
```

■ Simpy 코드

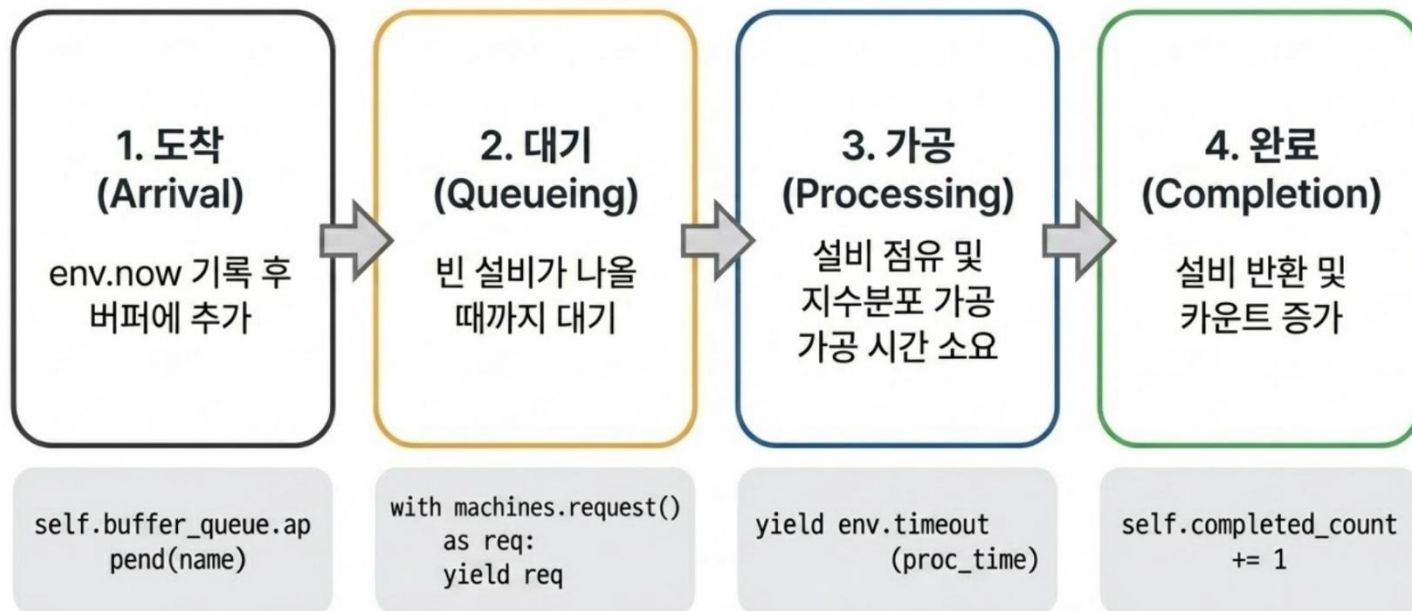
➤ simpy 핵심 code

<code>env = simpy.Environment()</code>	가상의 시뮬레이션 세계(엔진) 생성
<code>env.process(작업())</code>	백그라운드에서 특정 작업 지시 및 동시 실행
<code>yield env.timeout(N)</code>	지정된 N시간(가공/이동) 동안 대기하며 시간 전진
<code>simpy.Resource(capacity=N)</code>	동시에 N개만 사용할 수 있는 한정된 설비/자원 생성
<code>with 설비.request() as req:</code>	자원 점유 요청 (가득 찼을 경우 대기열 생성)

■ 스케줄링 예제 1 - 단일 부품 가공

➤ 단일 부품 가공(단일 공정) 프로세스

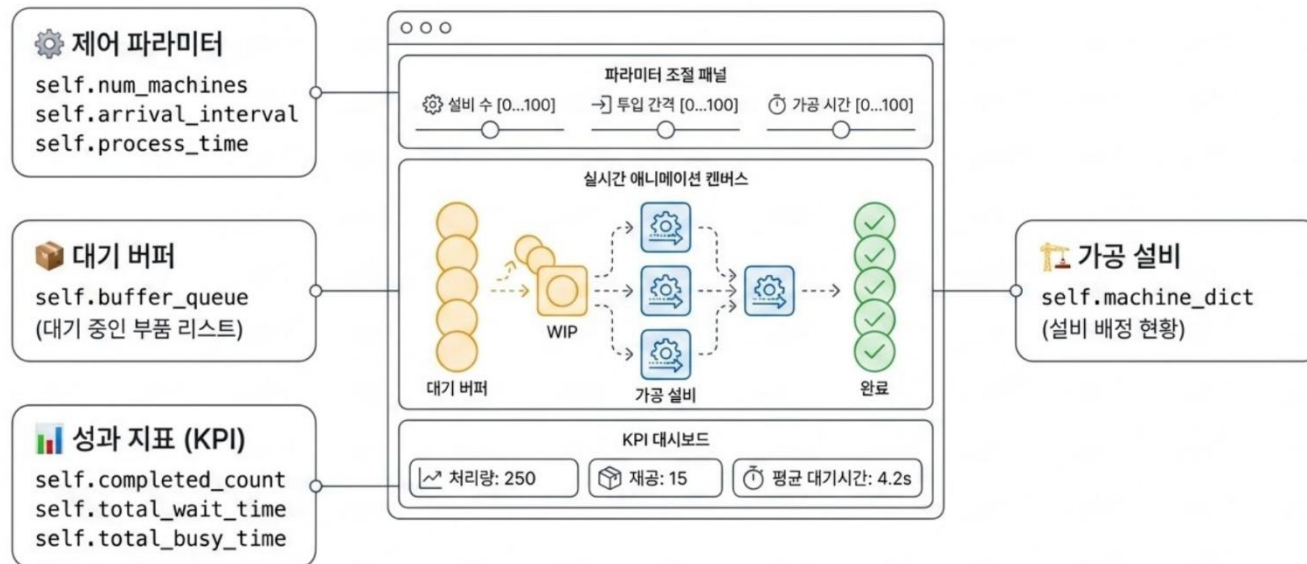
- 단일 작업 : 부품이 들어오면, 단 한 번의 가공 작업만 수행하고 바로 완료
- 동일 자원 : N대의 설비가 완전히 동일한 기능과 가공 속도를 가짐
- 단일 대기열 : 부품들은 하나의 버퍼에서 대기, N대 설비 중 비어있는 설비 배정



■ 스케줄링 예제 1 - 단일 부품 가공

➤ 단일 부품 가공(단일 공정) 프로세스

- 프론트엔드 - Tkinter GUI : 파라미터 입력 및 실시간 화면 렌더링[root.mainloop()]
- 백엔드 - Simpy 엔진 : 시간 흐름 통제 및 설비/부품 상태 연산[env.step()]



■ 스케줄링 예제 1 - 단일 부품 가공

➤ 단일 부품 가공(단일 공정) 프로세스

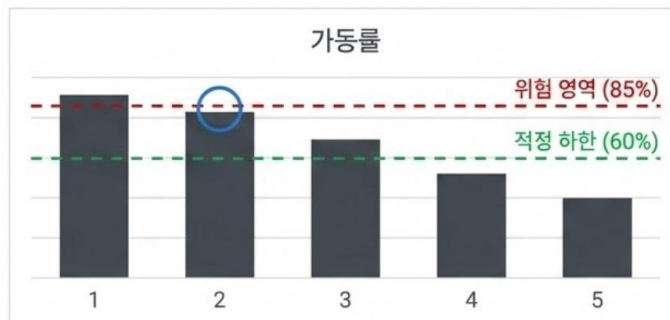
- 핵심 성과 지표 확인을 통한 최적의 공정 파라미터 도출

설비 수	생산량	처리량	평균대기	가동률	평균WIP	최대WIP
1대	166개	0.332/s	124.50초	99.6%	41.5개	83개 ← 병목!
2대	248개	0.496/s	4.20초	74.4%	2.1개	9개
3대	251개	0.502/s	0.80초	50.2%	0.4개	4개
4대	251개	0.502/s	0.10초	37.6%	0.0개	2개
5대	251개	0.502/s	0.02초	30.1% ← 과잉	0.0개	1개

■ 스케줄링 예제 1 - 단일 부품 가공

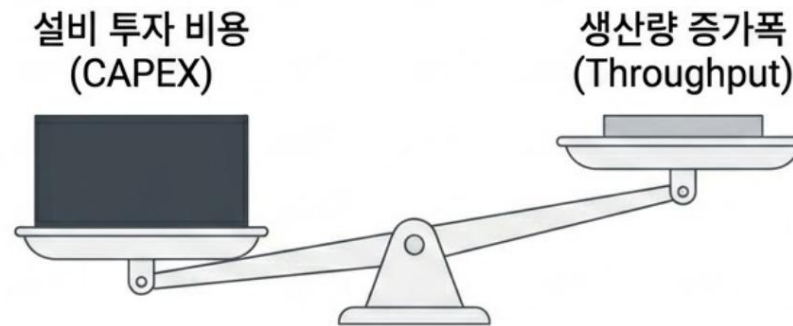
➤ 단일 부품 가공(단일 공정) 프로세스

- 핵심 성과 지표 확인을 통한 최적의 공정 파라미터 도출



■ 스케줄링 예제 1 - 단일 부품 가공

➤ 단일 부품 가공(단일 공정) 프로세스



Core Message: 설비를 무한정 늘리는 것이 답이 아니다.

Detail • 시뮬레이션 결과, 설비를 3대 이상으로 늘려도 처리량(Throughput)은 더 이상 유의미하게 증가하지 않으며 가동률만 40% 이하로 추락합니다.

Takeaway • 투자 비용 대비 효과(ROI)가 급감하는 '포화 지점'을 선제적으로 파악하는 것이 제조 데이터 분석의 핵심 목적입니다.

■ 스케줄링 예제 2 - JSSP

➤ Job Shop Scheduling Problem

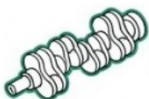
부품 마다 기계를 거치는 순서(Routing)가 제각각인 공정

Makespan : 모든 작업의 마지막 공정이 끝나는 시점

목표 : Makespan의 최소화



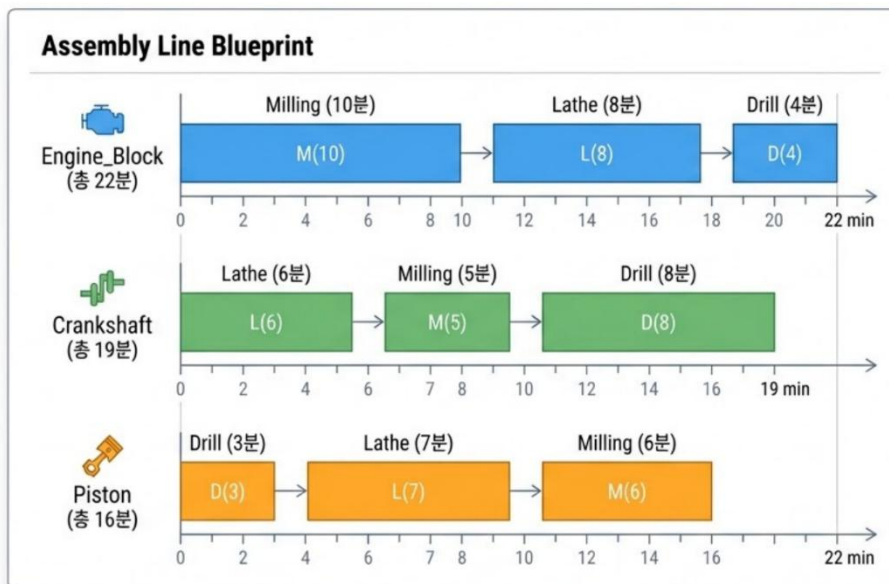
Engine Block



Crankshaft



Piston



Makespan (전체 완료 시간)

모든 작업의 마지막 공정이 끝나는 시점. 우리의 목표는 이 Makespan을 최소화하는 것입니다.

Job	Process (Time)
Engine_Block (총 22분)	Milling (10분) → Lathe (8분) → Drill (4분)
Crankshaft (총 19분)	Lathe (6분) → Milling (5분) → Drill (8분)
Piston (총 16분)	Drill (3분) → Lathe (7분) → Milling (6분)

■ 스케줄링 예제 2 - JSSP

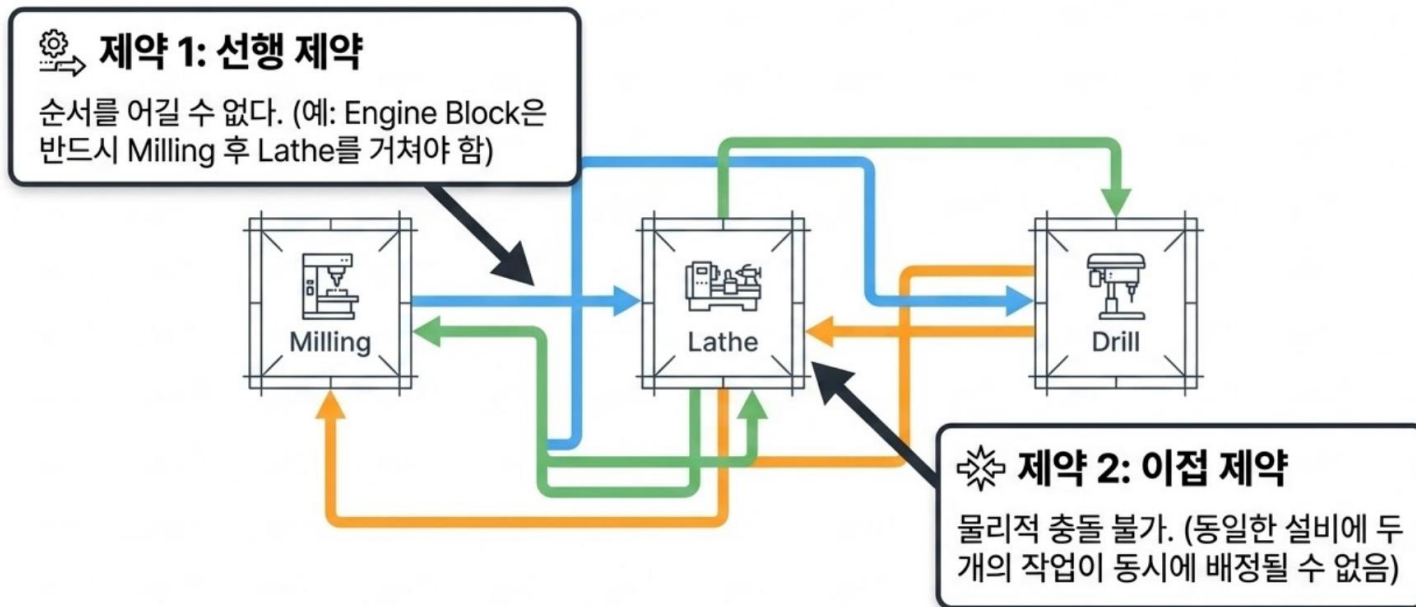
➤ Job Shop Scheduling Problem

JSSP의 제약 조건

작업 선후행 제약 : 각 작업은 사전에 정의된 고유한 공정(operation) 순서를 따라야 함

작업 비선점 제약 : 한 작업의 특정 공정이 일단 시작되었다면, 완료될 때까지 중단 불가

기계 용량 제약 : 하나의 기계는 동시에 단 하나의 작업만 처리



■ 스케줄링 예제 2 - JSSP

➤ MILP (혼합정수선형계획법) : 전역 최적해

$$\begin{aligned}
 &\text{Minimize } C_{\max} \\
 &\text{Subject to:} \\
 &S[j, m] \geq S[j, p] + T[j, p] \quad \text{for all } j \in \text{Jobs}, p \rightarrow m \in \text{Process} \\
 &C[j] = S[j, \text{last}] + T[j, \text{last}] \quad \text{for all } j \in \text{Jobs} \\
 &C_{\max} \geq C[j] \quad \text{for all } j \in \text{Jobs} \\
 &\sum_{p \text{ on } m} X[j, p, m, t] \leq 1 \quad \text{for all } m \in \text{Machines}, t \in \text{Time}
 \end{aligned}$$



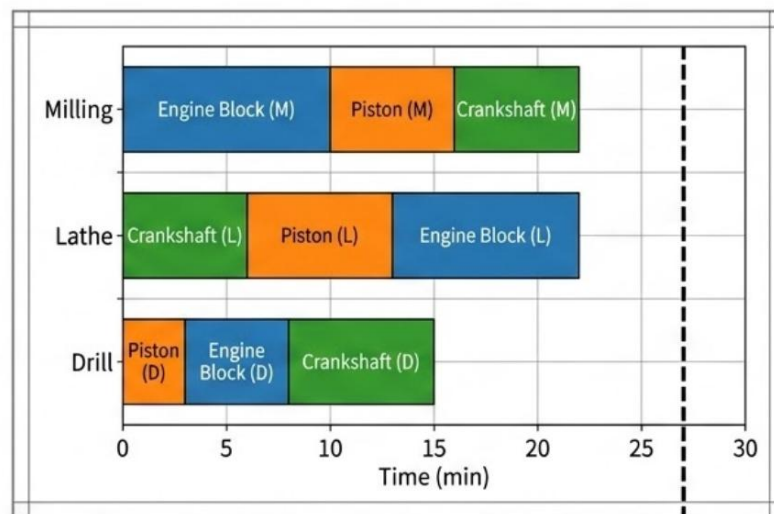
핵심 개념

문제를 수학적 공식(MILP)으로 변환하여 증명된 최적해를 도출합니다.



작동 원리

결정 변수(시작 시간)와 제약 조건(순서, 설비 중복 방지)을 통해 Makespan($C_{\max}$)을 최소화합니다.

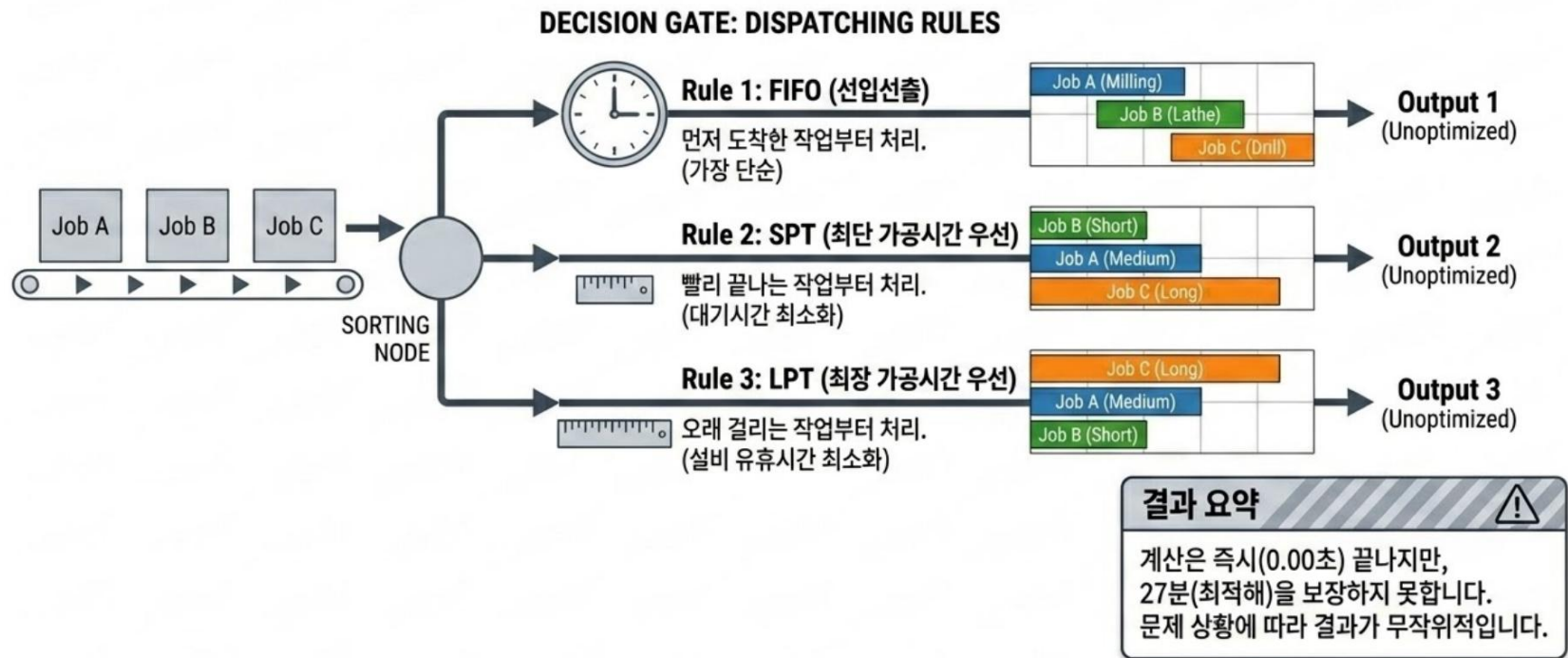


결과: 27분 (Ground Truth)

수학적으로 증명된 한계선입니다. 어떤 기법도 이보다 짧은 스케줄을 만들 수 없습니다.

■ 스케줄링 예제 2 - JSSP

➤ 휴리스틱(Dispatching Rules) : 직관적이고 즉각적인 경험 기반 규칙



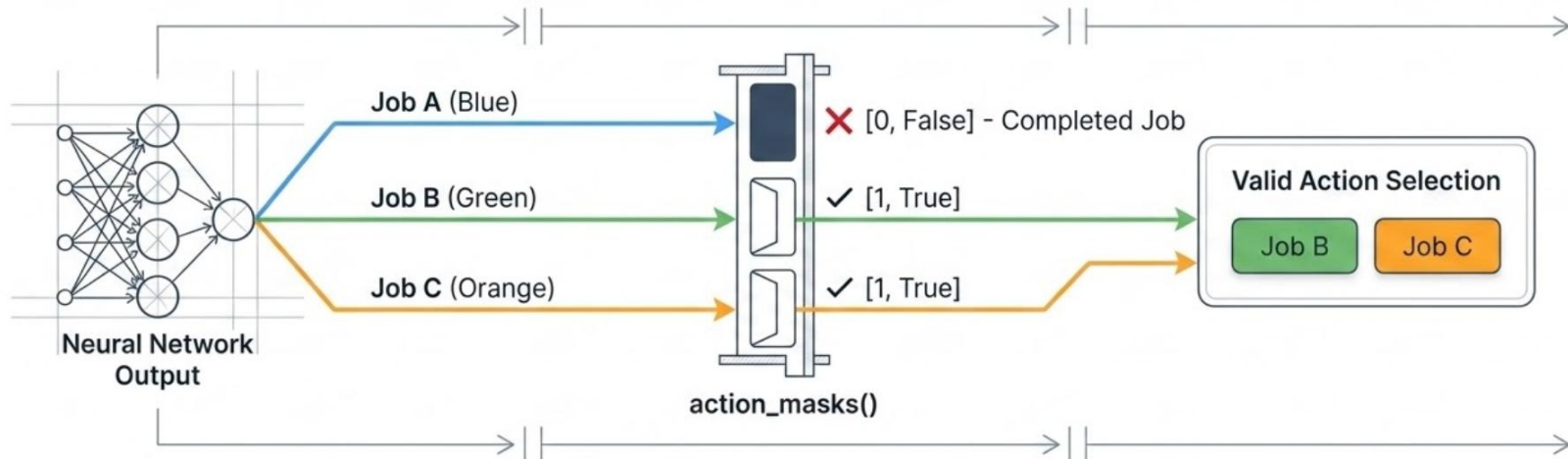
■ 스케줄링 예제 2 - JSSP

➤ 강화학습 : AI 스스로 스케줄링 규칙 학습



■ 스케줄링 예제 2 - JSSP

➤ 강화학습+(action masking) : AI의 선택지 제한



핵심 개념

도메인 지식으로 AI의 선택지를 물리적으로 제한합니다.

작동 원리

완료된 작업은 선택지에서 아예 제거(False 반환)합니다.
Maskable PPO는 오직 실행 가능한(True) 작업 공간 내에서만 확률을 계산합니다.

적용 효과

Invalid Action 발생률 0%.
AI가 쓸데없는 탐색에 낭비하는 시간을 없애고 최적화에 집중하게 만듭니다.

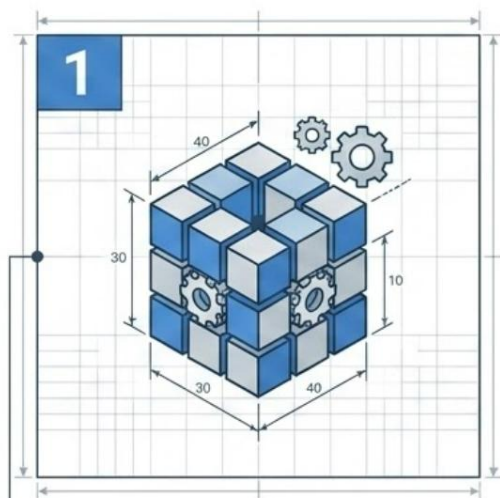
■ 스케줄링 예제 2 - JSSP

➤ 어떤 스케줄링 기법을 선택해야할까?

기법	최적 보장성	연산 속도	최적 적용 환경
MILP	⊙ 완벽 보장 (O)	매우 느림	소규모의 정적(Static) 환경
휴리스틱	⊗ 미보장 (X)	즉시 (ms)	실시간 빠른 의사결정이 필요한 현장
Maskable DRL	확률적 근사	학습 후 즉시	대규모의 동적(Dynamic) 환경

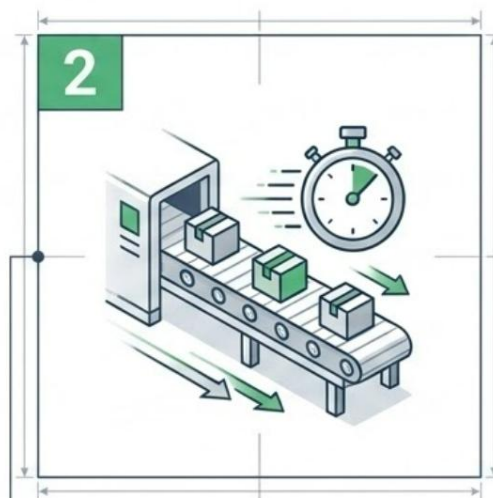
■ 스케줄링 예제 2 - JSSP

➤ 제조 분야에서 상황에 맞는 알고리즘 설계



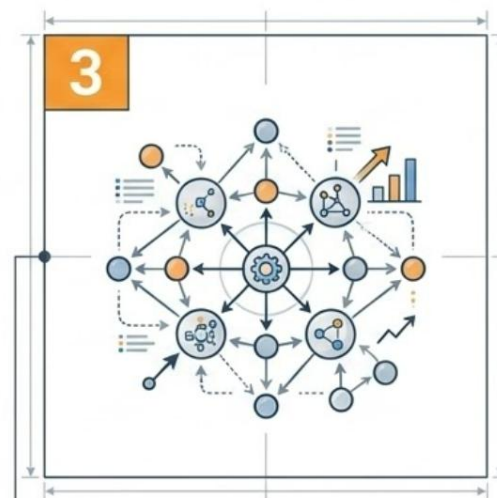
1. 정적이고 규모가 작은 공정

ILP를 도입하십시오.
타협 없는 정확한 최적해(Ground Truth)를 얻을 수 있습니다.



2. 1분 1초가 급한 실시간 현장

휴리스틱(SPT/LPT)을 배치하십시오.
완벽하진 않아도 멈춤 없는
즉각적 의사결정이 가능합니다.



3. 변동성이 크고 복잡한 대규모 환경

Maskable PPO (DRL)를 구축하십시오.
도메인 지식(Action Masking)으로 뼈대를 잡고,
자가 학습으로 변화에 적응하는
가장 강력한 AI 스케줄러가 됩니다.

감사합니다

Q&A

